

Workout Tracker PWA —

Architecture Brief

Purpose: A private, installable workout tracker for two users (owner + one invited user). Runs on iPhone via Safari home-screen install. No App Store, no Mac, no recurring cost.

Guiding constraints:

1. **Zero recurring cost.** Everything must sit inside a free tier that does not require a card to start.
2. **Secure by default.** Auth is passkey-first. Data is isolated per user at the database level, not just in application code.
3. **Small surface area.** Two users. Do not build for scale that will never arrive.

Stack

Layer	Choice	Why
Framework	Vue 3 + TypeScript (Composition	Matches existing skill set; Vue 3 for current

Layer	Choice	Why
	API, <code><script setup></code>)	tooling and better TS inference
Build	Vite + <code>vite-plugin-pwa</code>	Generates manifest and service worker; sane defaults for offline shell
State	Pinia	Minimal, TS-native
Backend	Supabase (Postgres + Auth + RLS)	Auth, database, and row-level authorization in one free tier
Hosting	Cloudflare Pages	Free static hosting, custom domain optional, no cold starts
Email (fallback only)	Supabase built-in, or Resend free tier if rate limits bite	Only used for account recovery

Explicitly not using: React Native, Expo, Capacitor, any native build path. No Apple Developer account. No

server-side rendering. No custom Node backend.

Authentication

Model

Passkey-first (WebAuthn), with magic link retained as a recovery path only.

Enrollment flow:

1. User signs in once via magic link to establish the account.
2. Immediately prompt to register a passkey.
3. Subsequent sign-ins use the passkey (Face ID on iPhone, one tap).

Magic link is never the primary path after enrollment. It exists so a lost credential does not mean a lost account.

Requirements

- Verify Supabase's current WebAuthn support before building. It is a newer feature and the API may have changed. If it is not production-ready at implementation time, fall back to magic-link-only auth and layer passkeys in later. Do not hand-roll a WebAuthn implementation.

- Register **at least two passkeys** per account where possible, or confirm iCloud Keychain sync is enabled so credentials survive device loss.
- Test passkey registration **inside the installed home-screen PWA context**, not only in a Safari tab. Behavior has historically differed between the two.
- Set a long session expiry. Safari can evict storage during idle periods, which forces re-auth.

Access control

The app is invite-only. There is no public sign-up.

- Disable open sign-ups in the Supabase Auth settings.
- Maintain an allowlist: either seed the two user rows manually, or gate account creation on an email present in an `allowed_users` table.
- Do not rely on "nobody knows the URL" as a control.

Data model

Keep it narrow. Resist adding tables until a real need appears.

```

profiles
  id          uuid  PK, references
auth.users(id)
  display_name text
  created_at   timestampz

exercises
  id          uuid  PK
  user_id     uuid  FK -> profiles(id)
  name        text
  category    text      -- e.g.
push / pull / legs / cardio
  is_archived boolean default false

workouts
  id          uuid  PK
  user_id     uuid  FK -> profiles(id)
  performed_at timestampz
  notes       text

sets
  id          uuid  PK
  workout_id  uuid  FK -> workouts(id) on
delete cascade
  exercise_id uuid  FK -> exercises(id)
  set_index   int      -- ordering
within the workout
  reps        int
  weight       numeric
  weight_unit  text      -- 'lb' |
'kg'
  rpe         numeric null

```

Notes:

- `weight_unit` is stored per set rather than as a global setting. Cheap now, avoids a migration later.
 - `exercises` is per-user rather than a shared global catalog. Two users, no benefit to sharing, and it keeps RLS uniform.
 - Soft-delete exercises via `is_archived` so historical sets keep resolving to a name.
-

Row-level security

This is the actual security boundary. Application code is not.

Enable RLS on every table. No exceptions, including tables that seem harmless.

```
alter table profiles enable row level
security;
alter table exercises enable row level
security;
alter table workouts enable row level
security;
alter table sets      enable row level
security;
```

Policy shape for directly-owned tables:

```
create policy "own rows only" on exercises
for all
using (auth.uid() = user_id)
with check (auth.uid() = user_id);
```

`sets` has no `user_id` of its own, so it inherits ownership through its workout:

```
create policy "own sets only" on sets
for all
using (
  exists (
    select 1 from workouts w
    where w.id = sets.workout_id
    and w.user_id = auth.uid()
  )
)
with check (
  exists (
    select 1 from workouts w
    where w.id = sets.workout_id
    and w.user_id = auth.uid()
  )
);
```

Verification step, not optional: after writing policies, sign in as user A and attempt to read and write user B's rows by ID directly against the API. Confirm both fail. A policy that was never tested against a hostile read is a policy you do not know works.

Client security

- Only the Supabase **anon** key ships to the browser. The service role key must never appear in client code, environment files committed to the repo, or the build output. It bypasses RLS entirely.
 - Put secrets in Cloudflare Pages environment variables, not in the repo.
 - Set a Content Security Policy. Restrict `connect-src` to the Supabase project URL and self.
 - Enforce HTTPS. Cloudflare Pages does this by default; do not disable it.
 - Validate input on write. RLS controls *who* can write, not *what* they can write. Add Postgres check constraints for sane ranges (`reps > 0` , `weight >= 0`).
-

PWA configuration

- `display: standalone` , portrait orientation, iPhone-appropriate theme color.
- Provide the full iOS icon set including `apple-touch-icon` . iOS ignores parts of the web manifest and needs its own meta tags.
- Service worker caches the app shell for offline load. Workout logging in a gym with poor signal is

the common case, so the shell must load without network.

- **Offline writes:** queue mutations in IndexedDB and sync when connectivity returns. This is the single most valuable feature for real gym use. Build it early rather than retrofitting.
 - iOS gives no install prompt. Add a one-time in-app hint explaining the Share → Add to Home Screen flow.
 - Add a data export (JSON or CSV download). Cheap insurance against storage eviction or a Supabase project pause.
-

Operational notes

- **Supabase pauses free projects after roughly a week of inactivity.** Resuming is a dashboard click, but if usage is sporadic this will be the main friction. Consider a scheduled ping if it becomes a nuisance, or accept it.
 - Free tier limits change. Verify current allowances at build time rather than trusting this document.
 - No analytics, no third-party scripts, no tracking. Two users, private fitness data, nothing to gain from instrumentation.
-

Build order

1. Vite + Vue 3 + TS scaffold, deployed to Cloudflare Pages. Confirm the pipeline works before writing features.
2. Supabase project, schema, and RLS policies. Test the isolation attack described above.
3. Auth: magic link first (it is the recovery path regardless), then passkey enrollment on top.
4. Core logging: create workout, add sets, view history.
5. PWA manifest, icons, service worker, install hint.
6. Offline queue and sync.
7. Data export.

Ship after step 4 if it is usable. The remaining steps improve it but are not blockers to real use.

Open decisions

- Whether exercise history needs charting, or whether a simple list of previous sets per exercise is enough. Start with the list.
- Whether the two users should ever see each other's workouts. Assume no; RLS above enforces

that. Changing it later means new policies, not a schema change.